问题：

如果想把一串序列颠倒过来要用到哪种数据结构。

栈。

解决下棋的悔棋问题要用到哪种数据结构？

栈。

第3章 栈和队列

3.1 栈

3.2 栈的应用举例

3.3 栈与递归的实现

3.4 队列

栈和队列是两种特殊的线性表。

特殊在哪？

本章重点难点

重点：(1)栈、队列的定义、特点、性质和应用；

(2)栈、队列的设计和实现以及基本操作及相关算法。

难点：(1)循环队列；

(2)栈、队列的应用。

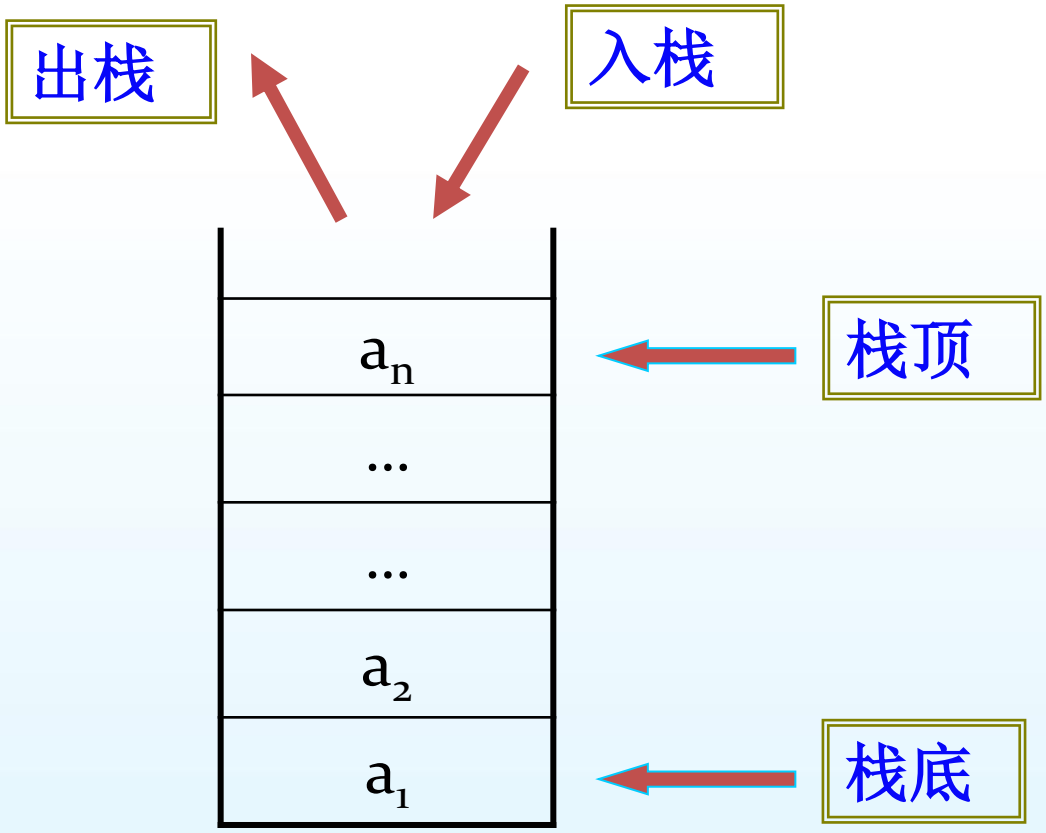
3.1 栈

3.1.1 抽象数据类型栈的定义

3.1.2 栈的表示和实现

3.1.1 抽象数据类型栈的定义

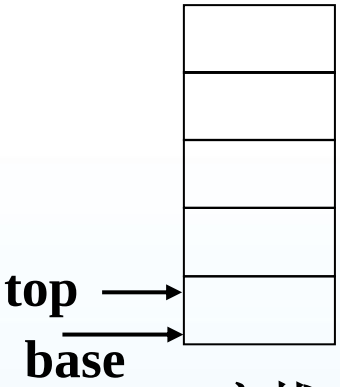
- 栈的示意图



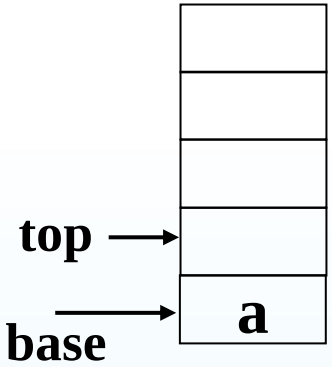
- 栈的特点

后进先出(Last In First Out，简称LIFO)。又称栈为后进先出表(简称LIFO结构)。

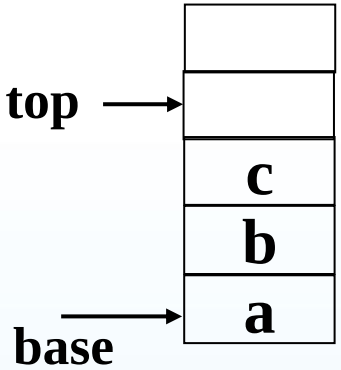
堆栈变化示意图



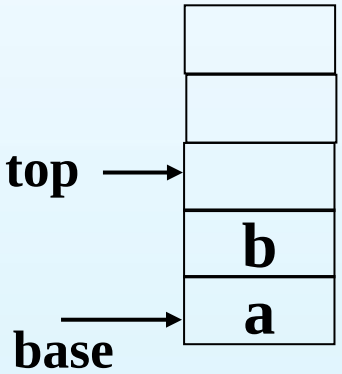
空栈



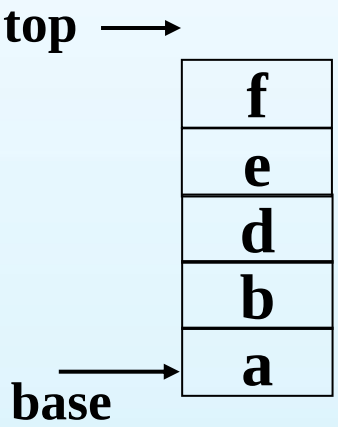
元素a进栈



元素b，c进栈



元素c退栈



元素d，e，f进栈

3.1.1 抽象数据类型栈的定义

□栈的定义

栈(Stack)是一种特殊的线性表，其插入和删除操作均在表的一端进行，是一种运算受限的线性表。

□栈的术语

栈顶(top)是栈中允许插入和删除的一端。

栈底(bottom)是栈顶的另一端。

3.1.1 抽象数据类型栈的定义

□抽象数据类型栈

ADT Stack {

数据对象： $D = \{ a_i \mid a_i \in ElemSet, i=1,2,\ldots,n, \ n \geq 0 \}$

数据关系： $R1 = \{ <a_{i-1}, a_i> \mid a_{i-1}, a_i \in D, i=2,\ldots,n \}$

约定a_n端为栈顶，a_1端为栈底。

基本操作： 见下页

} ADT Stack

3.1.1 抽象数据类型栈的定义

- 栈的基本操作

InitStack(&S)	//初始化栈
DestroyStack(&S)	//销毁栈
ClearStack(&S)	//清空栈
StackEmpty(S)	//判栈空
StackLength(S)	//求栈长度
GetTop(S, &e)	//取栈顶元素
Push(&S, e)	//入栈
Pop(&S, &e)	//出栈
StackTravers(S, visit())	//遍历栈

3.1 栈

3.1.1 抽象数据类型栈的定义

3.1.2 栈的表示和实现

2.2 线性表的顺序表示和实现

□顺序表示的实现

回顾线性表的实现

```
#define  LIST_INIT_SIZE     80
                // 线性表存储空间的初始分配量
#define  LISTINCREMENT    10
                // 线性表存储空间的分配增量
typedef  struct {

     ElemType *elem;     // 存储空间基址

      int   length;          // 当前长度

      int   listsize;        // 当前分配的存储容量
                             // (以sizeof(ElemType)为单位)

} SqList;                    // 俗称 顺序表
```

3.1.2 栈的表示和实现

□顺序栈的C语言实现

```
//----- 栈的顺序存储表示 -----
#define  STACK_INIT_SIZE  100; //栈容量
#define  STACKINCREMENT   10;  //栈增量
typedef struct {
  SElemType  *base;    //基地址
  SElemType  *top;     //栈顶指针
  int  stacksize;      //栈容量
} SqStack;
SqStack  S;
```

3.1.2 栈的表示和实现

□栈初始化过程演示

(1) 给栈S申请栈空间

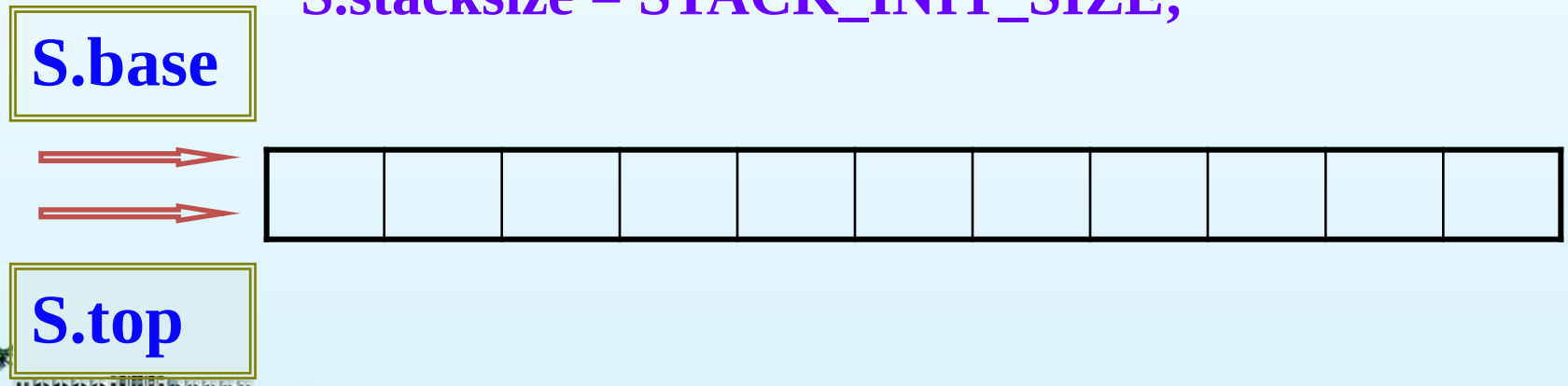
```
S.base=(SElemType*)malloc(STACK_INIT_SIZE*sizeof(SElemType));
```

(2) 设置栈顶地址S.top和基地址S.base的关系

```
S.top = S.base;
```

(3) 设置栈容量

```
S.stacksize = STACK_INIT_SIZE;
```



3.1.2 栈的表示和实现

□栈初始化算法

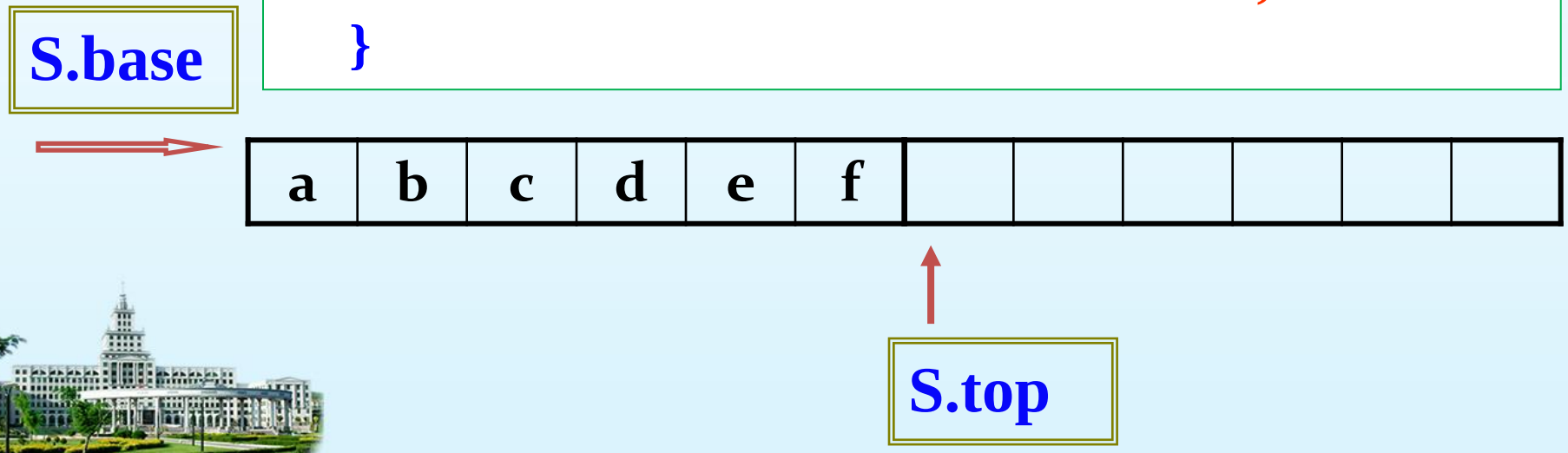
```
Status InitStack (SqStack &S){
                                        // 构造一个空栈S
 S.base=(SElemType*)malloc(STACK_INIT_SIZE*sizeof(SElemType));
  if (!S.base) exit (OVERFLOW); //存储分配失败
  S.top = S.base;
  S.stacksize = STACK_INIT_SIZE;
  return OK;
}
```

3.1.2 栈的表示和实现

□入栈操作演示

(1) 如果栈满，给栈增加容量

```
if (S.top - S.base >= S.stacksize)
 { S.base = (SElemType *) realloc ( S.base,
    (S.stacksize + STACKINCREMENT) *
                                  sizeof (SElemType));
   if (!S.base) exit (OVERFLOW); //存储分配失败
   S.top = S.base + S.stacksize;
   S.stacksize += STACKINCREMENT;
 }
```

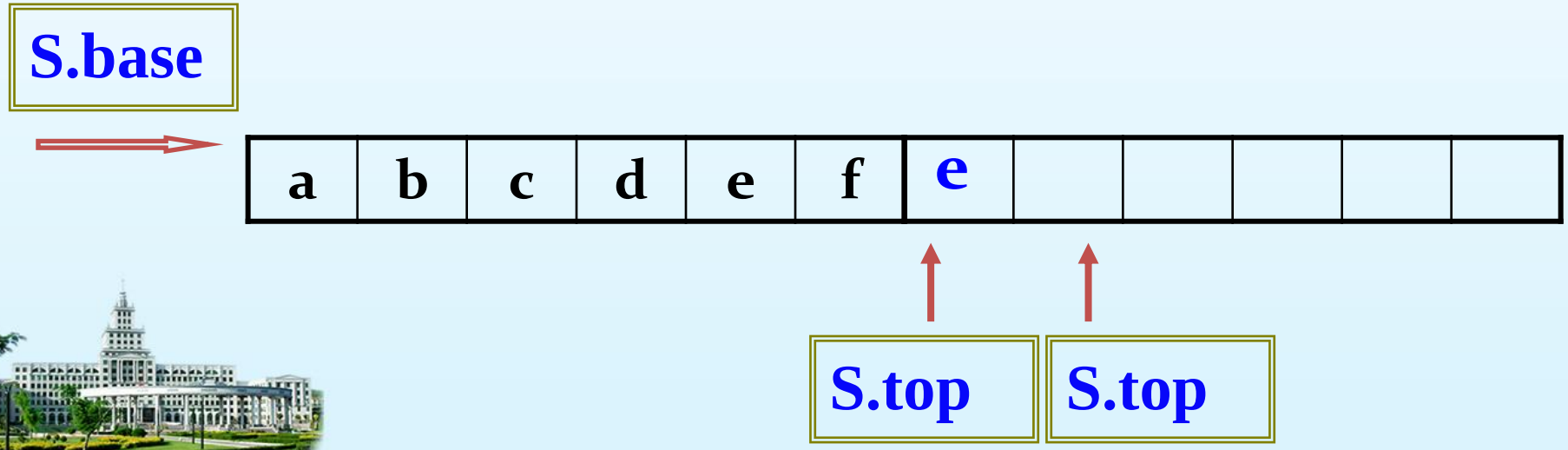


3.1.2 栈的表示和实现

□入栈操作演示

(2) 将数据存入栈顶位置，栈顶后移一位

*S.top = e;
S.top++



3.1.2 栈的表示和实现

□入栈操作演示

```
Status Push (SqStack &S, SElemType e) {
  if (S.top - S.base >= S.stacksize)     {
      S.base = (ElemType *) realloc ( S.base,
       (S.stacksize + STACKINCREMENT) * sizeof (ElemType));
      if (!S.base) exit (OVERFLOW); //存储分配失败
      S.top = S.base + S.stacksize;
      S.stacksize += STACKINCREMENT;
  }
  *S.top++ = e; return OK;
}
```

3.1.2 栈的表示和实现

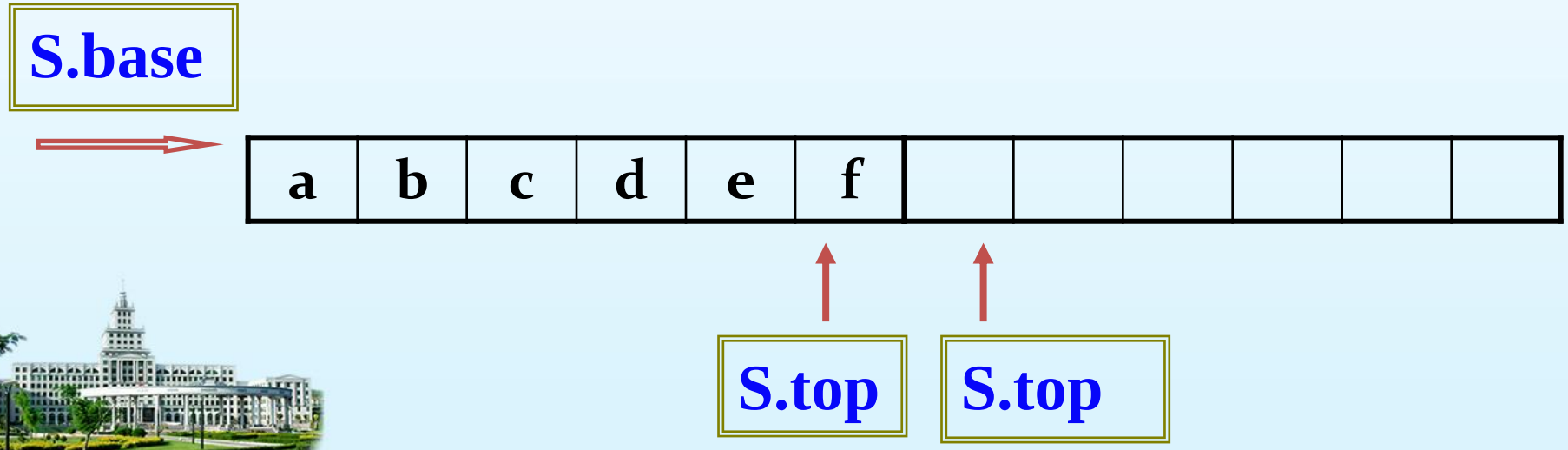
- 出栈操作演示

(1) 如果栈空，返回

```
if (S.top == S.base)
    return error;
```

(2) 否则：记录栈顶前面位置的值，栈顶位置减1

```
e=*(S.top-1);
S.top--;
```



3.1.2 栈的表示和实现

□出栈操作演示

```
Status Pop (SqStack &S, SElemType &e) {
    if (S.top ==S.base)
      return error;
    else
      {e=*(S.top-1);
       S.top--;}
      return OK;
}
```

3.1.2 栈的表示和实现

□其它栈操作讨论

DestroyStack(&S) //销毁栈

ClearStack(&S) //清空栈

StackEmpty(S) //判栈空

StackLength(S) //求栈长度

GetTop(S, &e) //取栈顶元素

StackTravers(S, visit()) //遍历栈

单选题 1分

设置

设1、2、…、n–1、n共n个数按顺序入栈，若第一个出栈的元素是n，则第三个出栈的元素是：

A 3

B n-3

C n-2

D 以上都可以

提交

单选题 1分

设置

已知入栈次序是12345，判断哪种出栈次序不对

A 12345

B 21345

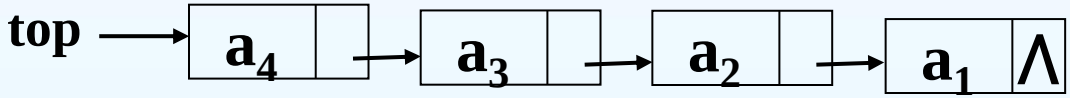
C 14235

D 21435

提交

3.1.2 栈的表示和实现

- 栈的链式存储结构称为链栈，是运算受限的单链表。
- 插入和删除操作只能在表头位置上进行。
- 栈顶指针top就是链表的头指针。



□链栈的C语言类型定义

```
Typedef struct  SNode {
    SElemType      data;      //数据域
    struct Snode *next;      //链域
}SNode, *LinkStack;
```

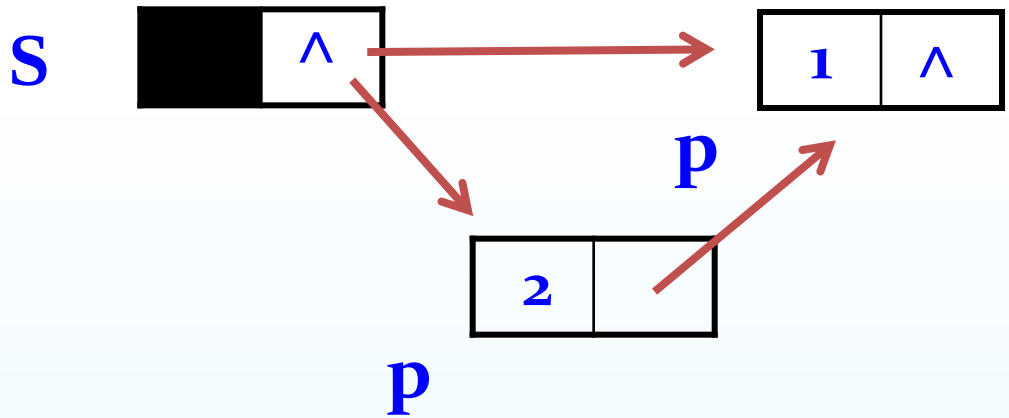
3.1.2 栈的表示和实现

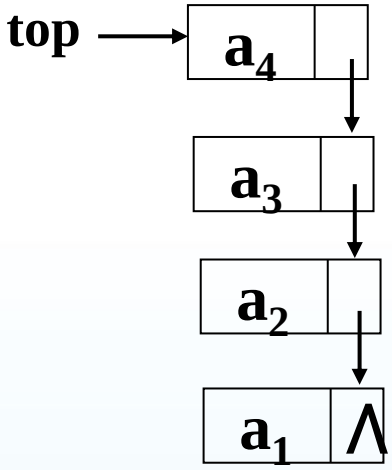
- 链栈初始化

```
LinkStack    S;

void InitStack(LinkStack &S){

 S=(SNode *) malloc (sizeof (SNode));
 if (!S)
 exit (OVERFLOW); //存储分配失败

 S->next=NULL;

 }
```

3.1.2 栈的表示和实现

□入栈操作:例如：入栈1和2





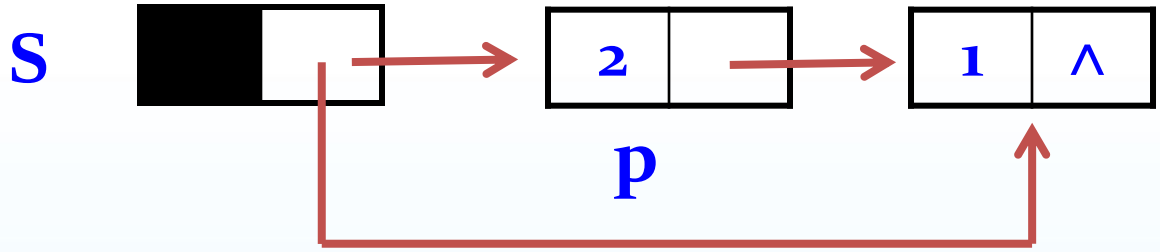
非空栈

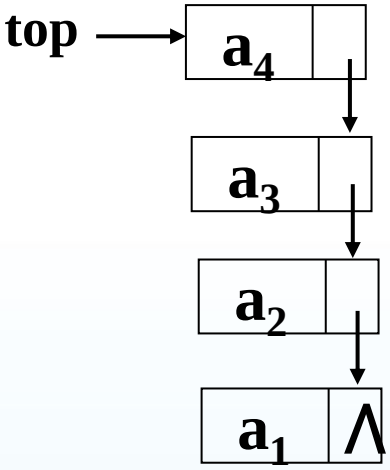
```
void  Push(LinkStack &S, SElemType e){

    p=(SNode *) malloc (sizeof (SNode));
    p->data=e;
    p->next=S->next;
    S->next=p;

}
```

链栈的入栈操作与哪种建立单链表的方式一致？

3.1.2 栈的表示和实现

□出栈操作:





非空栈

```
void  Pop(LinkStack &S, SElemType &e){
    if (S->next==NULL) return error;
    p=S->next;
    e=p->data;

    S->next=p->next;
    free(p);
}
```

链栈的出栈操作等于删除单链表的哪一个元素？

3.1.2 栈的表示和实现

- 讨论链栈基本操作的实现

InitStack(&S)	//初始化栈
DestroyStack(&S)	//销毁栈
ClearStack(&S)	//清空栈
StackEmpty(S)	//判栈空
StackLength(S)	//求栈长度
GetTop(S, &e)	//取栈顶元素
Push(&S, e)	//入栈
Pop(&S, &e)	//出栈
StackTravers(S, visit())	//遍历栈。

单选题 1分

设置

若用单向链表实现一个堆栈，当前链表状态为：1->2->3。当对该堆栈执行pop()、push(4)操作后，链表状态变成怎样?　　（1）4->2->3　（2）1->2->4

A 只能是（1）

B 只能是（2）

C （1）和（2）都有可能

D （1）和（2）都不可能

提交

单选题 1分

设置

在设计链栈时，通常采用单链表作为链栈，而不采用双链表作为链栈，其准确的原因是（ ）。

A 栈中元素是顺序存取的，用单链表就足够了

B 栈中元素是随机存取的，用单链表就足够了

C 双链表运算较单链表更复杂

D 双链表存储密度较单链表低

提交

3.2 栈的应用举例

3.2.1 数制转换

3.2.2 括号匹配的检验

3.2.3 迷宫求解

3.2.4 表达式求值

3.2.1 数制转换

□进制转换原理：

任何X进制数N转换成Y进制数其结果都是要化成如下形式。

$$N_X = a_n \times Y^n + \ldots + a_2 \times Y^2 + a_1 \times Y + a_0 \times Y^0$$

转换的过程就是通过取余运算求出$a_0,a_1,\ldots,a_n$，而先求出的是个位，十位…，与我们写数的习惯先从高位写正好相反，通过栈先进后出的特点，很容易实现倒过来的过程。

3.2.1 数制转换

例　将10进制1348转换成8进制

过程如下：

计算顺序

N	N div 8	N mod 8
1348	168	4
168	21	0
21	2	5
2	0	2

输出顺序

3.2.1 数制转换

□10进制数N转换成8进制的算法

```
void conversion () {
    InitStack(S); scanf ("%d",N);
    while (N) {
        Push(S, N % 8);
        N = N/8;
    }
    while (!StackEmpty(S)) {
        Pop(S,e);
        printf ( "%d", e );
    }
} // conversion
```

3.2 栈的应用举例

3.2.1 数制转换

3.2.2 括号匹配的检验

3.2.3 迷宫求解

3.2.4 表达式求值

3.2.2 括号匹配的检验

□问题描述

一个表达式中，包含三种括号“(”和“)”，“[”和“]”和“{”和“}”，这三种括号可以按任意的合法次序使用。

设计算法检验表达式中所使用括号的合法性。

□问题讨论

讨论：如何设计数据结构？

当遇到左括号时，进栈，遇到右括号时出栈。

3.2.2 括号匹配的检验

- 算法过程

(1)读入字符串，当遇到左括号时，进栈，遇到右括号时出栈；

(2) 当遇到某一个右括号时，栈已空，说明到目前为止，右括号多于左括号；

(3) 从栈中弹出的左括号与当前检验的右括号类型不同，说明出现了括号交叉情况；

(4) 算术表达式输入完毕，但栈中还有没有匹配的左括号，说明左括号多于右括号。

3.2.2 括号匹配的检验

□括号匹配检验算法

```
Status  check( ) {
   char ch;
   InitStack(S);
   while ((ch=getchar())!=‘#') {
      switch (ch) {
         case (ch=='(‘ || ch=='[‘ || ch=='{'):Push(S,ch);break;
         case (ch== ')'):
             if (StackEmpty(S)) retrun FALSE;
             else
                {  Pop(S,e);
                   if(e!= '(‘)
                      return FALSE;}
            break;
```

3.2.2 括号匹配的检验

- 括号匹配检验算法

```
 case (ch== ‘]'):
   if (StackEmpty(S))      retrun FALSE;
   else
   {      Pop(S,e);     if(e!= ‘[‘)     return FALSE;     }
    break;
    ............
   default:break;
}
}
if (StackEmpty(S))          return TRUE;
else     return FALSE;
}
```

3.2 栈的应用举例

3.2.1 数制转换

3.2.2 括号匹配的检验

3.2.3 迷宫求解

3.2.4 表达式求值

3.2.3 迷宫求解

□问题描述

如图表示一个迷宫，有一个入口，一个出口，从一个位置可以向8个方向走，0表示能走通，1表示走不通，求从入口到出口的路径。

入口 →

0	1	1	1	0	1	1	1
1	0	1	0	1	0	1	0
0	1	0	0	1	1	1	1
0	1	1	1	0	0	1	1
1	0	0	1	1	0	0	0
0	1	1	0	0	1	1	0

← 出口

3.2.4 迷宫求解

□问题描述

	0	1	2	3	4	5	6	7
0	-1	1	1	1	-1	1	1	1
1	1	-1	1	-1	1	-1	1	0
2	0	1	-1	-1	1	1	1	1
3	0	1	1	1	-1	-1	1	1
4	1	0	0	1	1	0	-1	-1
5	0	1	1	0	0	1	1	0

5	7
4	7
4	6
3	5
3	5
2	4
1	3
2	2
1	1
0	0

3.2.4 迷宫求解

- 用栈实现迷宫路径求解的过程

(1) 入口位置入栈，作标记以免重复进入。

(2) 从栈顶位置任选一个可以进的位置，并将此位置作标记避免重复，新进位置入栈。

(3) 重复(2)，若栈顶位置无路可走，则退栈，从新的栈顶重复(2)。

(4) 直到找到入口，从栈底到栈顶即是路径，或者栈空，表示从入口到出口没通路。

3.2.4 迷宫求解

- 四周墙壁解决办法如图

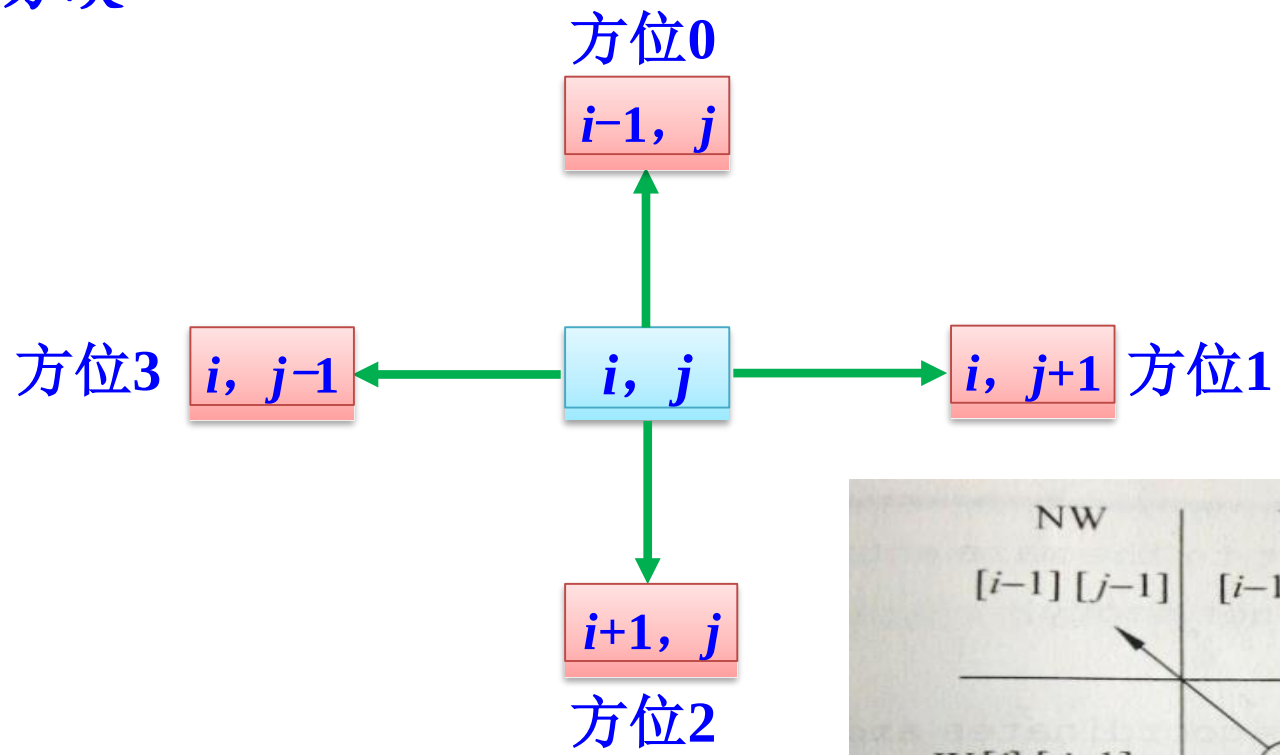
	0	1	2	3	4	5	6	7
0	0	1	1	1	0	1	1	1
1	1	0	1	0	1	0	1	0
2	0	1	0	0	1	1	1	1
3	0	1	1	1	0	0	1	1
4	1	0	0	1	1	0	0	0
5	0	1	1	0	0	1	1	0

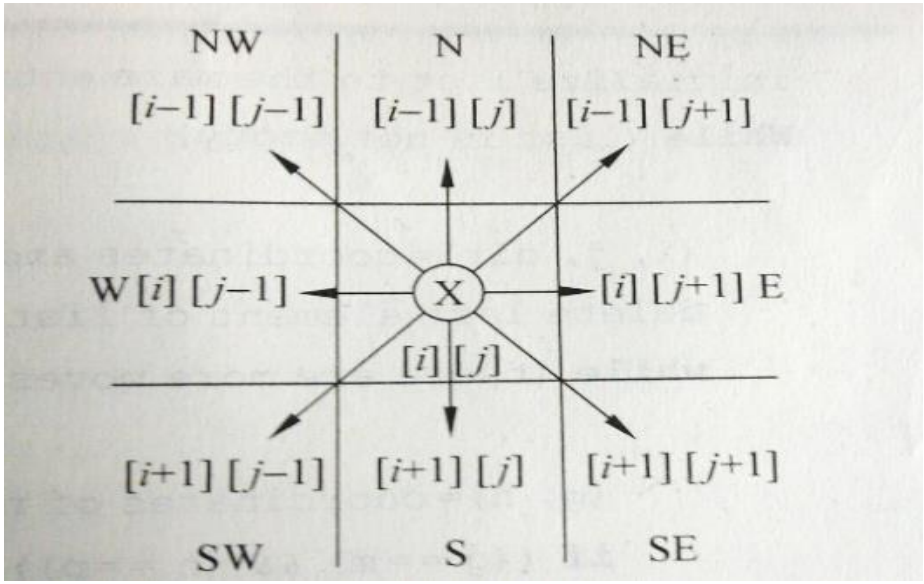
maze[6][8]

	0	1	2	3	4	5	6	7	8	9
0	1	1	1	1	1	1	1	1	1	1
1	1	0	1	1	1	0	1	1	1	1
2	1	1	0	1	0	1	0	1	0	1
3	1	0	1	0	0	1	1	1	1	1
4	1	0	1	1	1	0	0	1	1	1
5	1	1	0	0	1	1	0	0	0	1
6	1	0	1	1	0	0	1	1	0	1
7	1	1	1	1	1	1	1	1	1	1

maze[8][10]

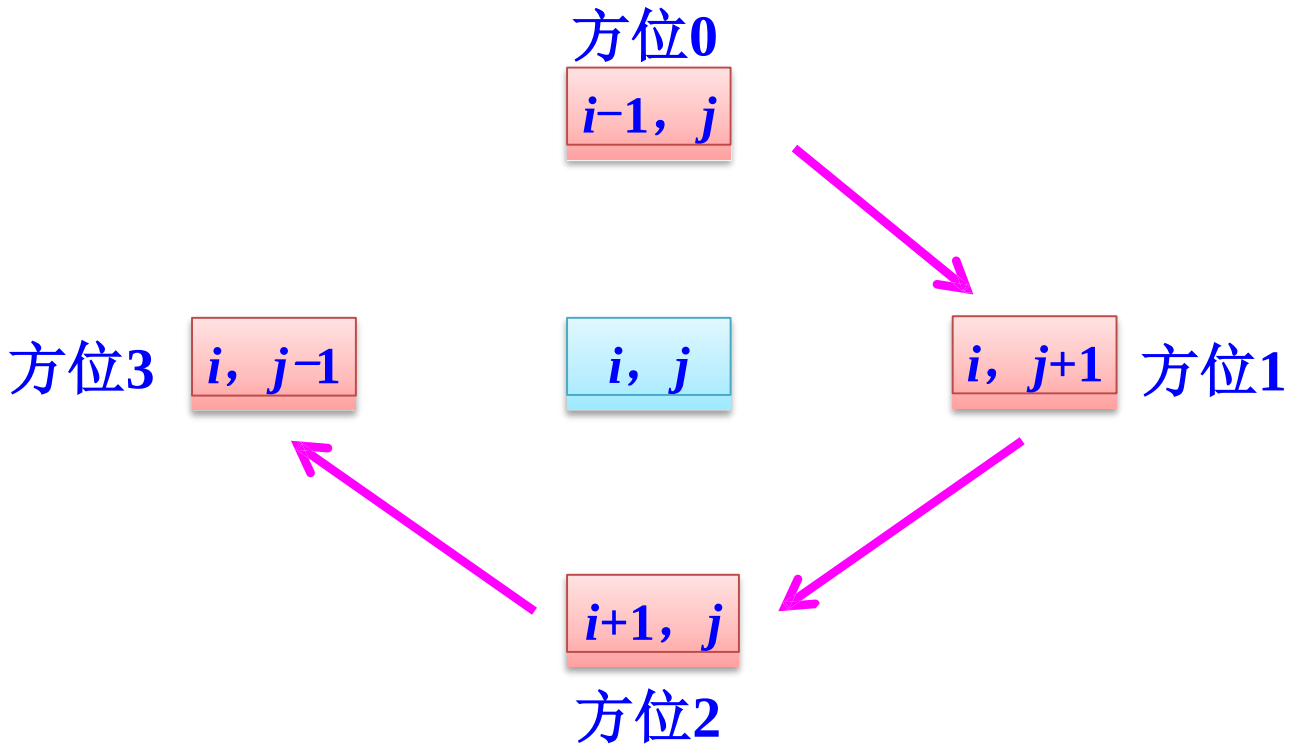
行走规则：上、下、左、右相邻方块行走。其中（*i*，*j*）表示一个方块



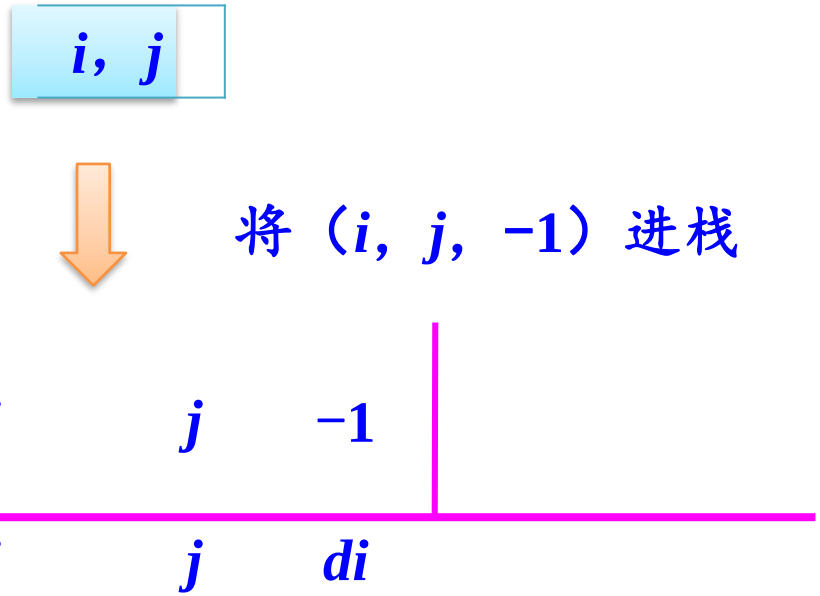


算法设计

试探顺序：从方位0开始，顺时针方向



初始时，入口（*i*，*j*）作为当前方块。



所有走过的方块都会进栈！

如果一个当前方块（i，j）找到一个相邻可走方块（x，y），就继续从（x，y）走下去。

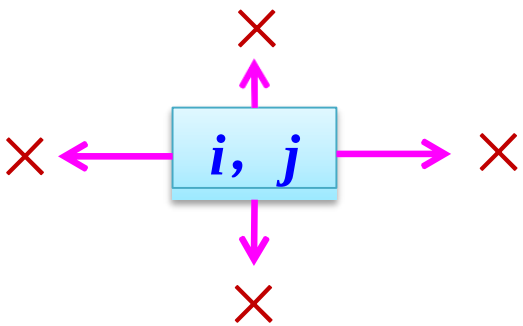
将（x，y，-1）进栈

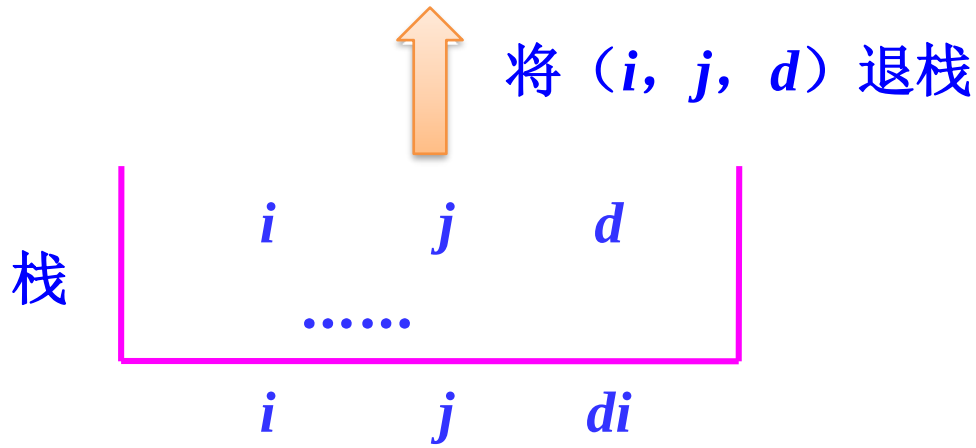
栈

x	y	-1
i	j	d

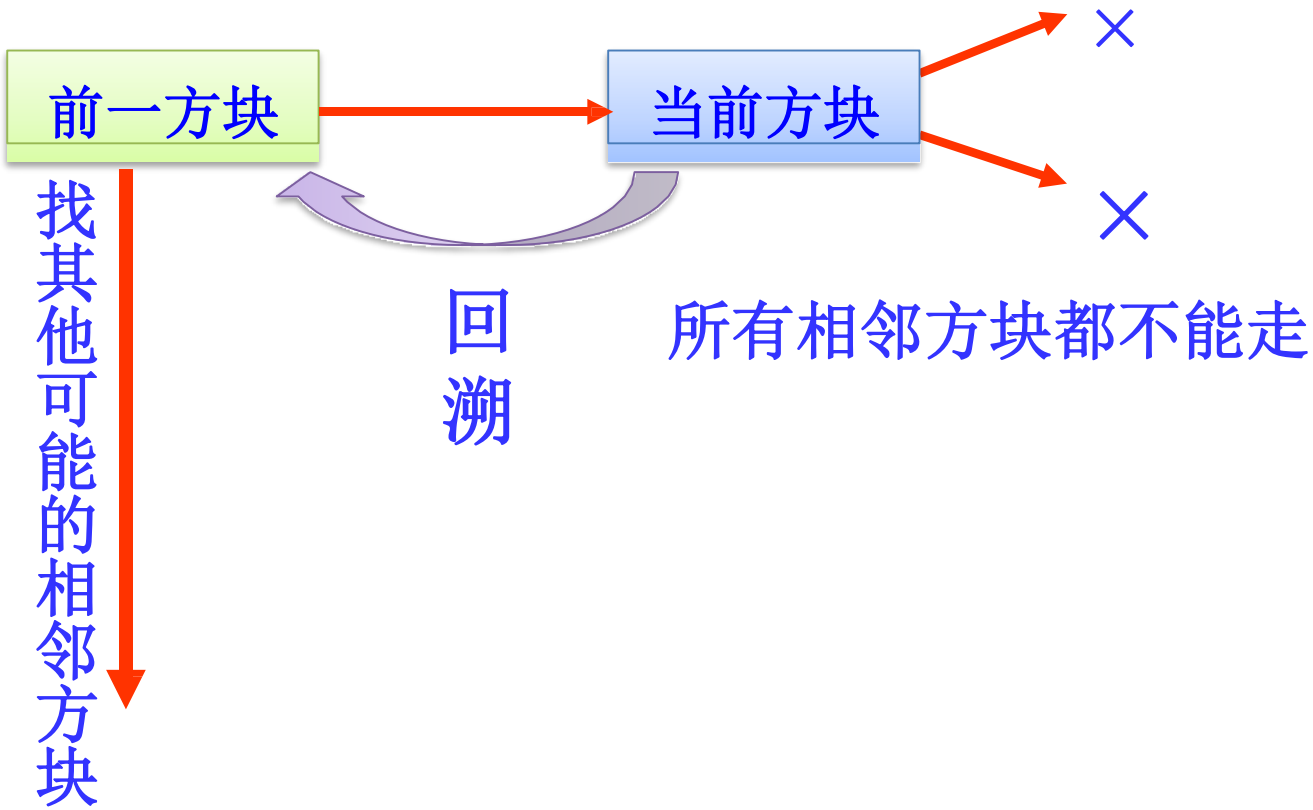
i　　j　　di

如果一个当前方块（*i*，*j*）没有找到任何相邻可走方块，表示此时无路可走，将其退栈。



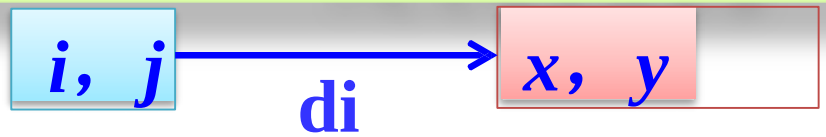


求解迷宫路径的过程：



在算法中用到的栈采用顺序栈存储结构，即将栈定义为：

```
typedef struct
{  int i;                    //当前方块的行号
   int j;                    //当前方块的列号
   int di;                   //di是下一可走相邻方位的方位号
} Box;
typedef struct               //定义方块类型
{   Box data[MaxSize];
    int top;                 //栈顶指针
} StType;                    //定义顺序栈类型
```



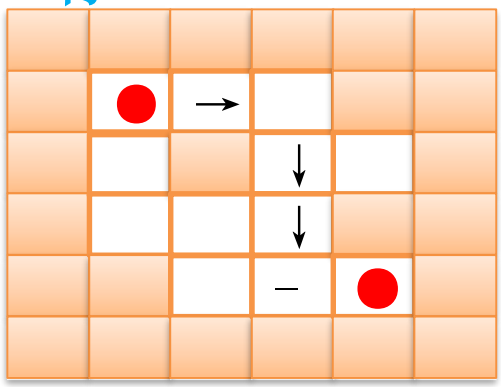
用栈求一条迷宫路径的算法：　（xi，yi）⇨（xe，ye）

```
bool mgpath(int xi,int yi,int xe,int ye)
{     int i,j,k,di,find;
      StType st;st.top=−1;        //定义栈st并初始化
      st.top++;                   //初始方块进栈
     st.data[st.top].i=xi;
     st.data[st.top].j=yi;
     st.data[st.top].di=-1;
     mg[xi][yi]=-1;
```

为了避免重复，当一个方块进栈时，将迷宫值 改为−1

```
while (st.top>-1)                    //栈不空时循环
{   i=st.data[st.top].i;
    j=st.data[st.top].j;
   di=st.data[st.top].di;            //取栈顶方块
   if (i==xe && j==ye)               //找到了出口,输出路径
   {   printf("迷宫路径如下:\n");
       for (k=0;k<=st.top;k++)
       {   printf("\t(%d,%d)",st.data[k].i,st.data[k].j);
           if ((k+1)%5==0)           //每输出每5个方块后换一行
           printf("\n");
       }
       printf("\n");
       return true;                  //找到一条路径后返回true
   }
```

i	*j*	di
4	4	−1
4	3	1
3	3	1
2	3	2
1	3	2
1	2	1
1	1	1

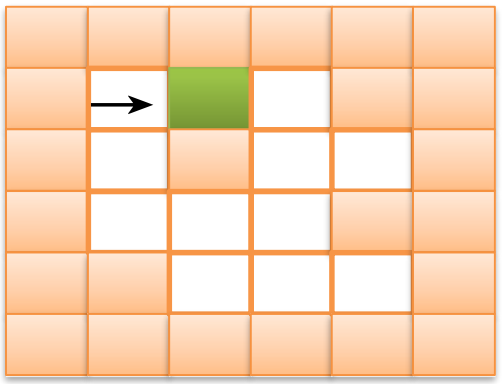


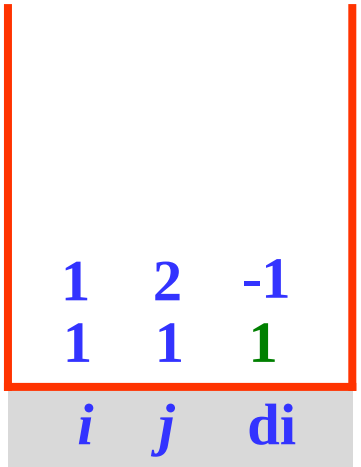
```
 find=0;
 while (di<4 && find==0)  //找下一个可走方块
{   di++;
    switch(di)
    {
    case 0:i=st.data[st.top].i-1; j=st.data[st.top].j;
            break;
    case 1:i=st.data[st.top].i; j=st.data[st.top].j+1;
            break;
    case 2:i=st.data[st.top].i+1; j=st.data[st.top].j;
            break;
    case 3:i=st.data[st.top].i, j=st.data[st.top].j-1;
            break;
    }
    if  (mg[i][j]==0) find=1;      //找到了下一个可走相邻方块(i，j)
 }//end of while
```

```
if (find==1)
{   st.data[st.top].di=di;
    st.top++;
    st.data[st.top].i=i;        st.data[st.top].j=j;
    st.data[st.top].di=-1;
    mg[i][j]=-1;
}
```

//找到了下一个可走方块
//修改原栈顶元素的di值
//下一个可走方块进栈
//避免重复走到该方块

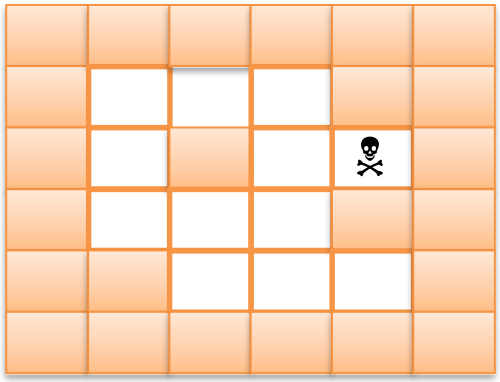
从入口（1,1）出发找到一个可走方块（1，2）: 将（1，2，-1）进栈

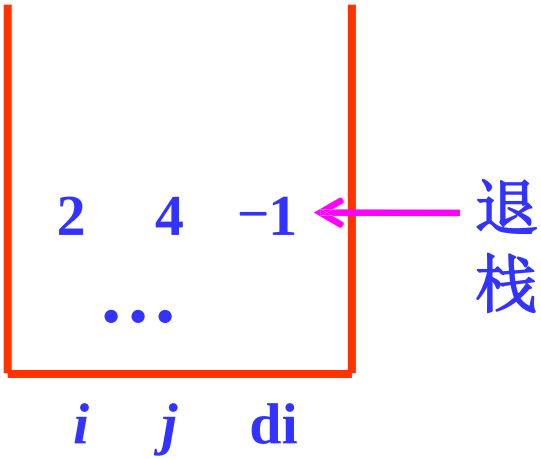




```
    else                        //没有路径可走,则退栈
    {    mg[st.data[st.top].i][st.data[st.top].j]=0;
                                //让该位置变为其他路径可走方块
         st.top--;              //将该方块退栈
    }

  }
  return false;        //表示没有可走路径,返回false

}
```

（2,4）方块没有通路





3.2 栈的应用举例

3.2.1 数制转换

3.2.2 括号匹配的检验

3.2.3 迷宫求解

3.2.4 表达式求值

3.2.4 表达式求值

□问题描述

一个表达式由操作数(operand)、运算符(operator)、界限符(delimiter)组成。写出“算符优先法”求值的算法。

例　求3*(2+3*5)+6的值

3.2.4 表达式求值

例　求3*(2+3*5)+6的值

θ1----栈顶符号

θ2----读入符号

θ1< θ2, 入栈

θ1> θ2, 操作

θ1= θ2, 操作

OPND

5
3 5
2 7
3

OPTR

*
+
(
*
#

3 * 5

2 + 15

3.2.4 表达式求值

□算法求解过程

设置两个栈，一个存操作数，栈名为OPND，一个存操作符，栈名为OPTR栈。

(1) 首先置操作数栈为空，表达式起始符#为运算符栈的栈底元素；

(2)依次读入表达式中每个字符，若是操作数则进OPND栈，若是运算符则和OPTR栈的栈顶运算符比较优先权后作相应操作，直到整个表达式操作完毕。

3.2.4 表达式求值

□算法求解过程

(1)若栈顶运算符小于输入运算符，输入运算符进栈OPTR；

(2)若栈顶运算符等于输入运算符（只有栈顶是“（”，输入是“）”，或者栈顶是“#”，输入是“#）”两种情况，分别去除一对括号，或结束。

(3)若栈顶运算符大于输入运算符，弹出栈顶运算符，从OPND中弹出两个操作数与弹出运算符计算后再存入OPND栈，继续。

3.2.4 表达式求值

- **表达式求值算法**

```
OperandType EvaluateExpression(){
   initStack(OPTR);//初始化操作符栈
  Push(OPTR,'#');
   initStack(OPND);//初始化操作数栈
   c=getchar();
   while(c!='#')||GetTop(OPTR)!='#'){
        if(!In(c,OP)) // 如果输入为数值，则入OPND栈
          {  Push((OPND,c);     c=getchar();   }
        else    //否则，根据操作符的优先级，选择进OPTR栈还是执行操作
          switch(Precede(GetTop(OPTR),c)){
```

```
    case '<': Push(OPTR,c);c=getchar();break;
    case '=': Pop(OPTR,x);c=getchar();break;
    case '>': Pop(OPTR,theta);
              Pop(OPND,a); Pop(OPND,b);
     Push(OPND,Operate(a,theta,b)); break;
    } //switch语句结束
} //while 语句结束
 return(GetTop(OPND));
} //算法结束
```

第3章 栈和队列

3.1 栈

3.2 栈的应用举例

3.3 栈与递归的实现

3.4 队列

3.3 栈与递归调用

◆递归调用的定义

- 子程序（或函数）直接调用自己或通过一系列调用语句间接调用自己。是一种描述问题和解决问题的基本方法。

◆递归的基本思想

- 把一个不能或不好求解的大问题转化为一个或几个小问题，再把这些小问题进一步分解成更小的小问题，直至每个小问题都可以直接求解。

◆递归的要素

- 递归边界条件：确定递归到何时终止，也称为递归出口；
- 递归模式：大问题是如何分解为小问题的，也称为递归体

3.3 栈与递归调用(Cont.)

◆递归调用举例

➢求阶乘的函数

$$n! = \begin{cases} 1 & \text{当 } n<=1 \text{ 时} \\ n * (n-1)! & \text{当 } n>1 \text{ 时} \end{cases}$$

终止条件

递推规则

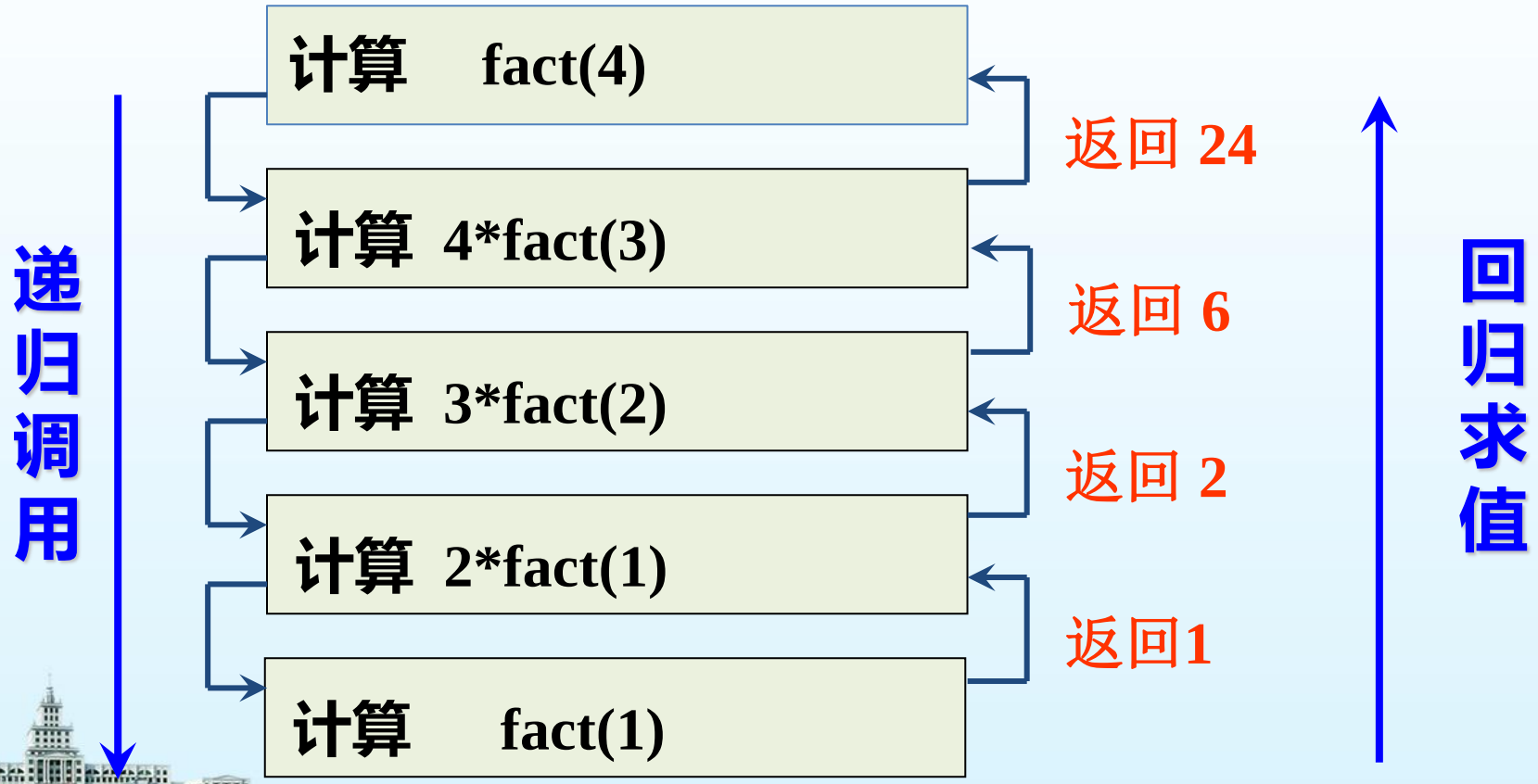
➢递归算法

```
long fact ( int n )
{
    if ( n <= 1 ) return 1;
    else  return  n * fact (n-1);
}
```

3.3 栈与递归调用(Cont.)

◆递归调用举例

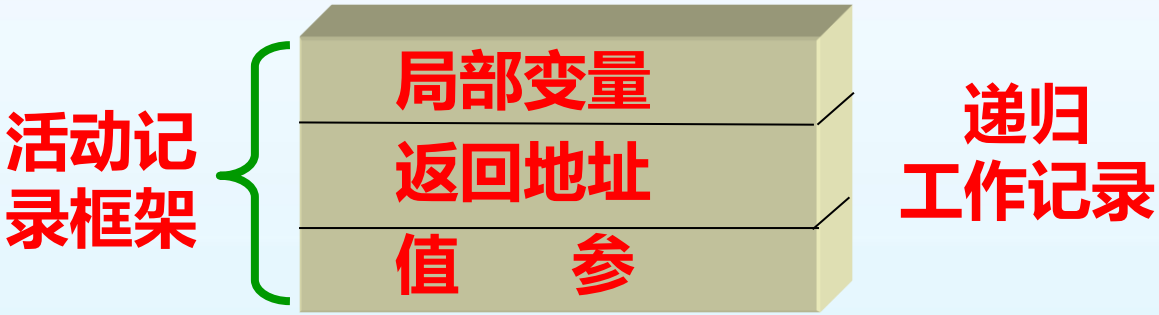
➢求解阶乘 n! 的过程



3.3 栈与递归调用(Cont.)

◆递归过程与递归工作记录

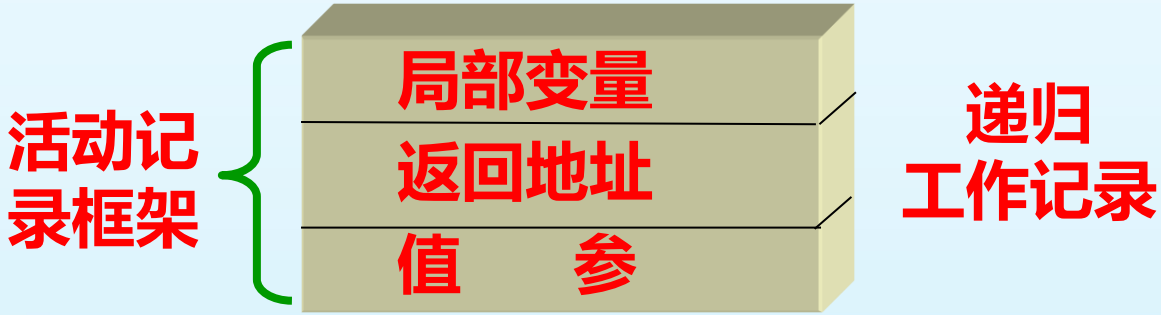
- 每一次递归调用时，需要为过程中使用的参数、局部变量和返回地址等另外分配存储空间。
- 每层递归调用需分配的空间形成递归工作记录，按后进先出的栈组织。
-



3.3 栈与递归调用(Cont.)

◆递归函数的内部执行过程

- (1) 运行开始时，首先为递归调用建立一个工作栈，其结构包括值参、局部变量和返回地址；
- (2) 每次执行递归调用之前，把递归函数的值参和局部变量的当前值以及调用后的返回地址压栈；
- (3) 每次递归调用结束后，将栈顶元素出栈，使相应的值参和局部变量恢复为调用前的值，然后转向返回地址指定的位置继续执行。



第3章 栈和队列

3.1 栈

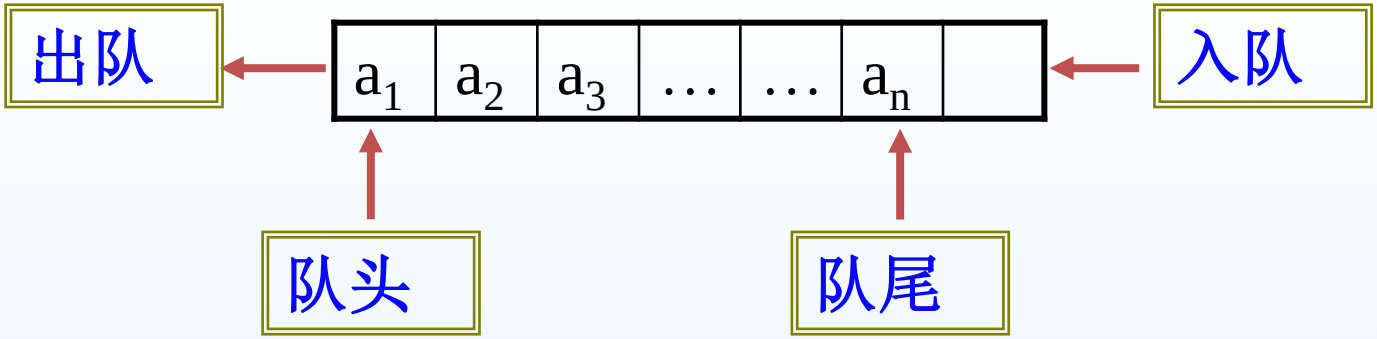
3.2 栈的应用举例

3.3 栈与递归的实现

3.4 队列

3.4.1 抽象数据类型队列的定义

□队列示意图



□队列的特点

先进先出(First In First Out， 简称FIFO)。

又称队列为先进先出表。

3.4.1 抽象数据类型队列的定义

□队列的定义

队列(Queue)——是一种运算受限的特殊的线性表，它只允许在表的一端进行插入，而在表的另一端进行删除。

□队列的术语

队尾(rear)是队列中允许插入的一端。

队头(front)是队列中允许删除的一端。

3.4.1 抽象数据类型栈的定义

□抽象数据类型队列

ADT Queue {

数据对象：$D = \{a_i \mid a_i \in ElemSet, i=1,2,...,n, n \geq 0\}$

数据关系：$R1 = \{ <a_{i-1}, a_i> \mid a_{i-1}, a_i \in D, i=2,...,n\}$

约定其中a_1端为队列头， a_n端为队列尾

基本操作：

见下页

} ADT Queue

3.4.1 抽象数据类型栈的定义

□队列的基本操作

InitQueue(&Q)	//初始化队列
DestroyQueue(&Q)	//销毁队列
QueueEmpty(Q)	//判队列是否空
QueueLength(Q)	//求队列长度
GetHead(Q, &e)	//取队头元素
ClearQueue(&Q)	//清空队列
EnQueue(&Q, e)	//入队列
DeQueue(&Q, &e)	//出队列
QueueTravers(Q, visit())	//遍历队列。

讨论：

设栈S和队列Q的初始状态为空，元素a、b、c、d、e、f依次通过栈S，一个元素出栈后即进入队列Q，若这6个元素出队列的顺序是b、d、c、f、e、a则栈S的容量至少应该是 _ _ _3_ _ _。

3.4.2 链队列

□链队列结点实现

```
typedef struct QNode { // 结点类型
    QElemType      data;
    struct QNode  *next;
  } QNode, *QueuePtr;
```

□链队列数据类型实现

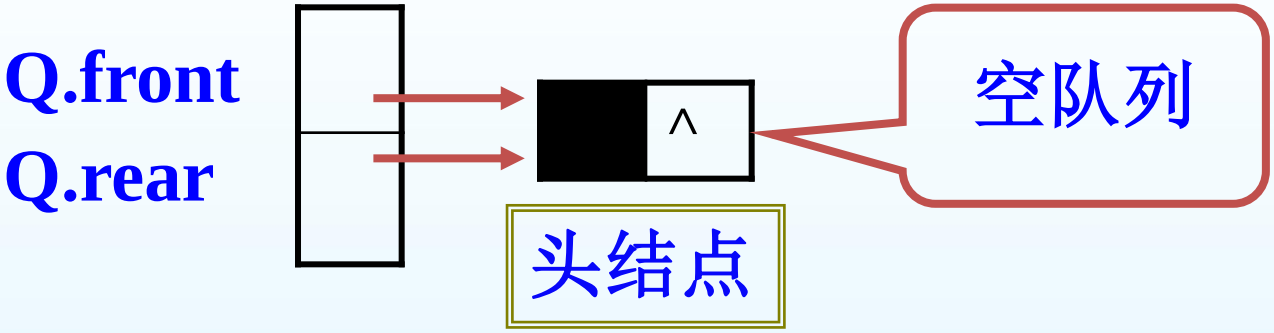
```
typedef struct {          // 链队列类型
    QueuePtr  front;      // 队头指针
    QueuePtr  rear;       // 队尾指针
} LinkQueue;
```

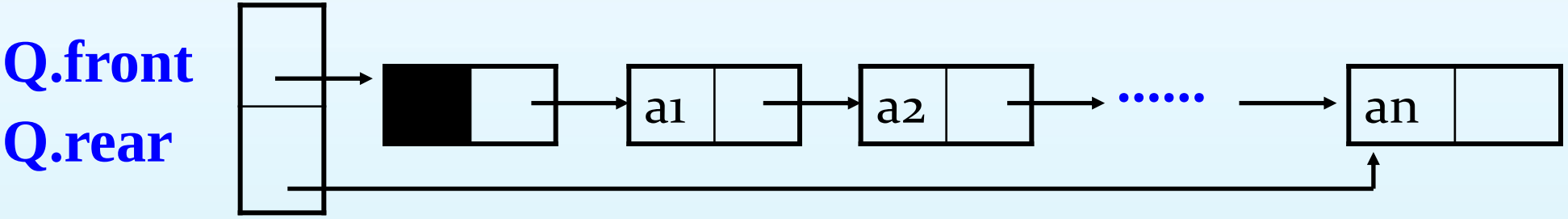
为什么链队列常用两个指针？

3.4.2 链队列

- 带头结点的链队列示意图

LinkQueue Q;





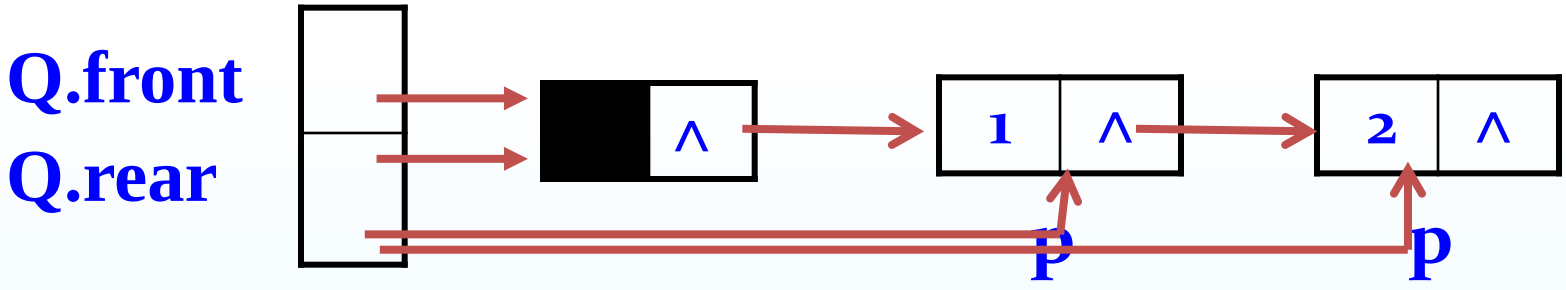
3.4.2 链队列

□带头结点的链队列初始化

```
Status InitQueue (LinkQueue &Q) {
                                    // 构造一个空队列Q
  Q.front = Q.rear =  (QueuePtr)malloc(sizeof(QNode));
  if (!Q.front)
    exit (OVERFLOW);
                                    //存储分配失败
  Q.front->next = NULL;
  return OK;
}
```

3.4.2 链队列

- 带头结点的链队列入队算法示意图：例如：1和2入队




```
p = (QueuePtr) malloc (sizeof (QNode));
if (!p)  exit (OVERFLOW);
p->data = e;   p->next = NULL;
Q.rear->next = p;
Q.rear = p;
```

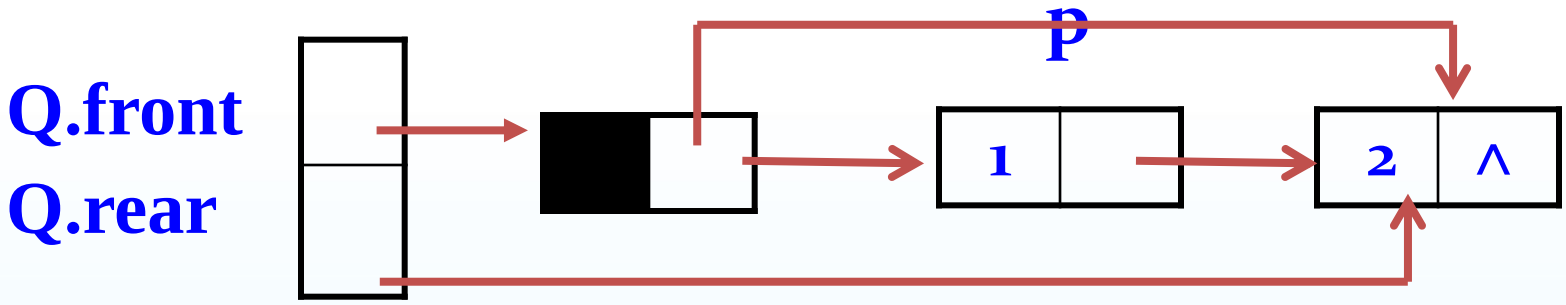
3.4.2 链队列

□带头结点的链队列入队算法

```
Status EnQueue (LinkQueue &Q, QElemType e)
{   // 插入元素e为Q的新的队尾元素
    p = (QueuePtr) malloc (sizeof (QNode));
    if (!p)
        exit (OVERFLOW);   //存储分配失败
    p->data = e;   p->next = NULL;
    Q.rear->next = p;    Q.rear = p;
    return OK;
}
```

3.4.2 链队列

- 带头结点的链队列出队算法示意图：




```
Status DeQueue (LinkQueue &Q, QElemType &e) {
  if (Q.front == Q.rear)    return ERROR;
  p = Q.front->next;   e = p->data;
  Q.front->next = p->next;
  if (Q.rear == p)  Q.rear = Q.front;
  free (p);      return OK;
}
```

单选题 1分

设置

在一个链表表示的队列中，f和r分别指向队列的头和尾。下列哪个操作能正确地将s结点插入到队列中:

A f->next=s; f=s;

B r->next=s; r=s;

C s->next=r; r=s;

D s->next=f; f=s;

提交

3.4.2 链队列

- 带头结点的链队列其它操作讨论

DestroyQueue(&Q)	//销毁队列
QueueEmpty(Q)	//判队列是否空
QueueLength(Q)	//求队列长度
GetHead(Q, &e)	//取队头元素
ClearQueue(&Q)	//清空队列
QueueTravers(Q, visit())	//遍历队列

3.4.3 循环队列

□顺序队列数据类型实现

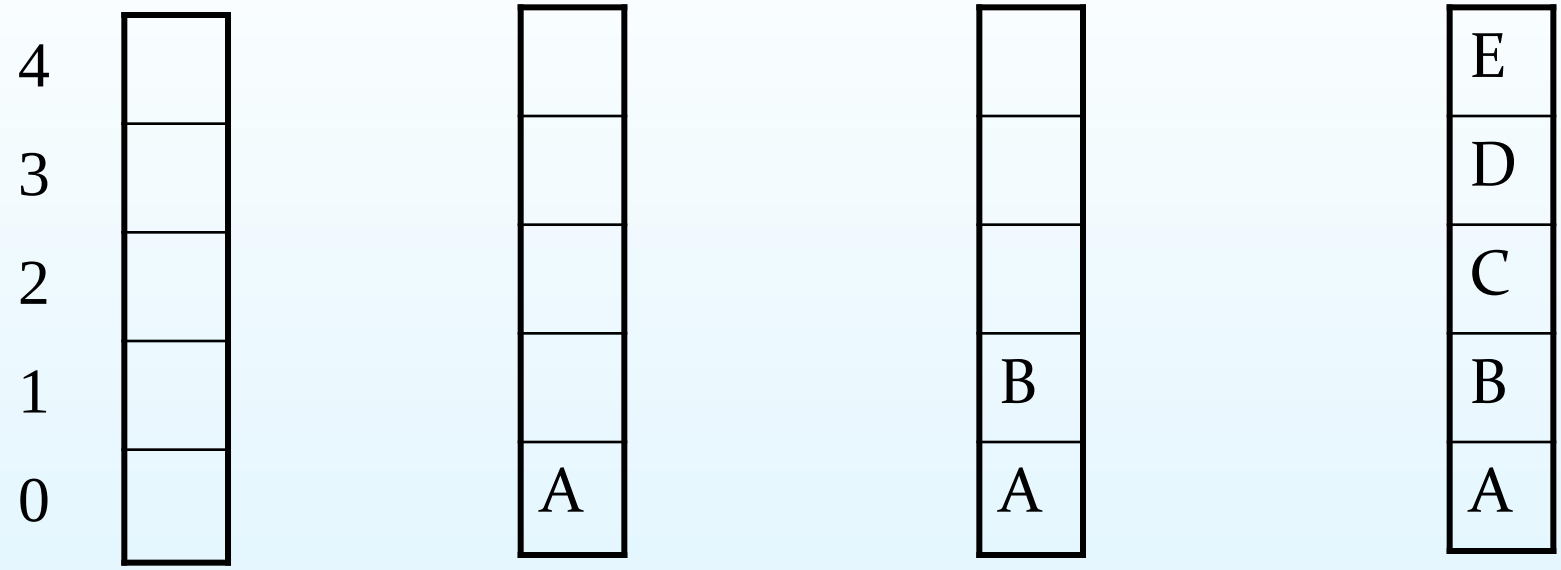
```
#define MAXQSIZE  100  //最大队列长度
typedef struct {
  QElemType  *base;  // 动态分配存储空间
  int  front;     // 头指针，若队列不空，指向队列头元素
  int  rear;     // 尾指针，若队列不空，指向
                          队列尾元素 的下一个位置
} SqQueue;

SqQueue    Sq;
```

3.4.3 循环队列

□顺序队列讨论

讨论在采用一般顺序队列时出现的问题。

入队过程



Sq.rear=0
Sq.front=0

Sq.rear=1
Sq.front=0

Sq.rear=2
Sq.front=0

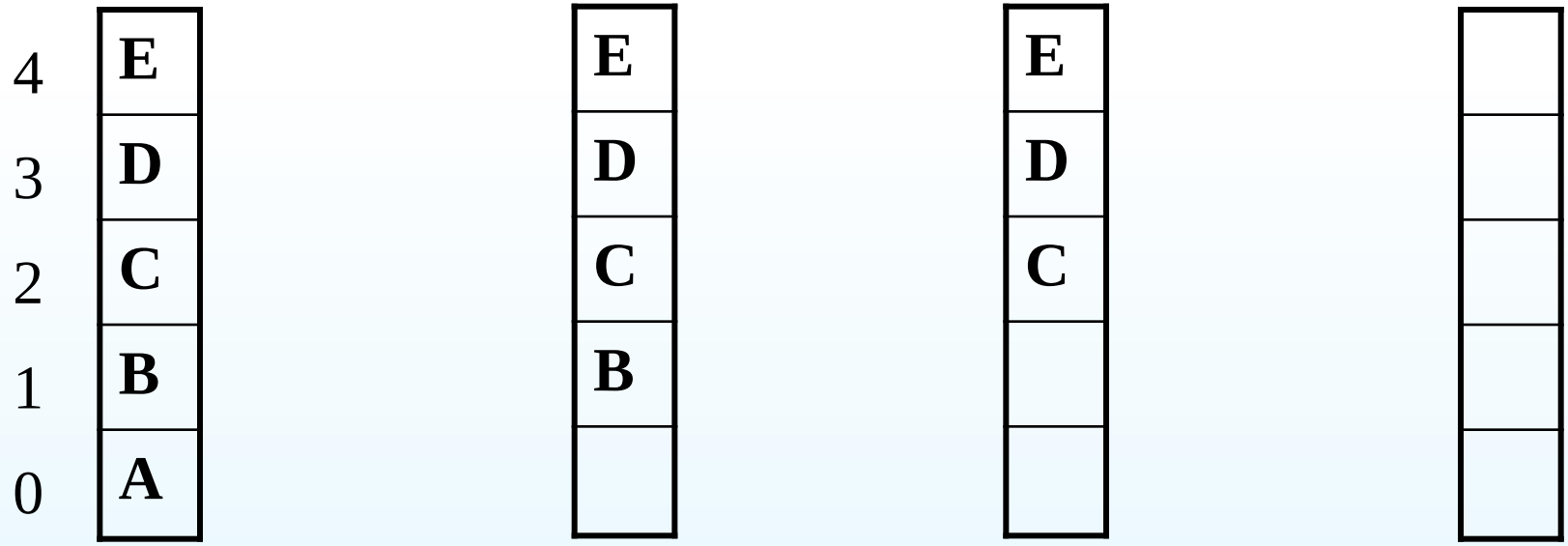
Sq.rear=5
Sq.front=0

rear== MAXQSIZE 作为队满条件

3.4.3 循环队列

□顺序队列讨论

出队过程



Sq.rear=5	Sq.rear=5	Sq.rear=5	Sq.rear=5
sq.front=0	Sq.front=1	Sq.front=2	Sq.front=5

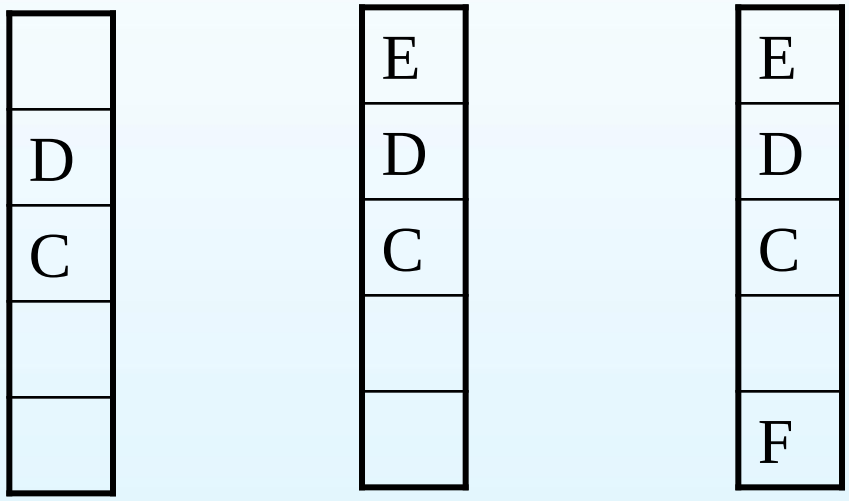
讨论结论：在采用一般顺序队列时出现假上溢现象。

这是因为采用 rear== MAXQSIZE 作为队满条件的缺陷 。 当队满条件为真时 队中可能还有若干空位置 。这种溢出并不是真正的 溢出 称为 假 溢出。

3.4.3 循环队列

□循环队列的定义

循环队列是顺序队列的一种特例，它是把顺序队列构造成一个首尾相连的循环表。指针和队列元素之间关系不变。



Sq.rear=4
Sq.front=2

Sq.rear=0
Sq.front=2

Sq.rear=1
Sq.front=2

3.4.3 循环队列

□循环队列数据类型实现

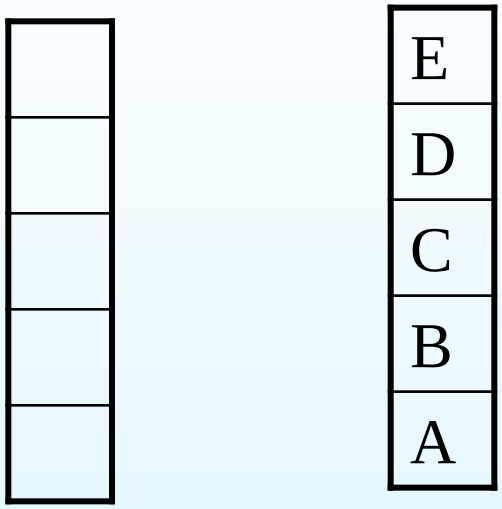
```
#define MAXQSIZE  100  //最大队列长度
 typedef struct {
   QElemType  *base;  // 动态分配存储空间
   int  front;     // 头指针，若队列不空，
                   //  指向队列头元素
   int  rear;     // 尾指针，若队列不空，指向
                     队列尾元素 的下一个位置
 } SqQueue;
SqQueue    Sq;
```

循环队列是顺序存储结构还是链式存储结构？

3.4.3 循环队列

□循环队列空状态和满状态的讨论

空状态　　满状态



Sq.rear=0　　**Sq.rear=0**

Sq.front=0　　**Sq.front=0**

讨论结论：循环队列空状态和满状态都满足

Q.front==Q.rear

3.4.3 循环队列

- 循环队列空状态和满状态的判别

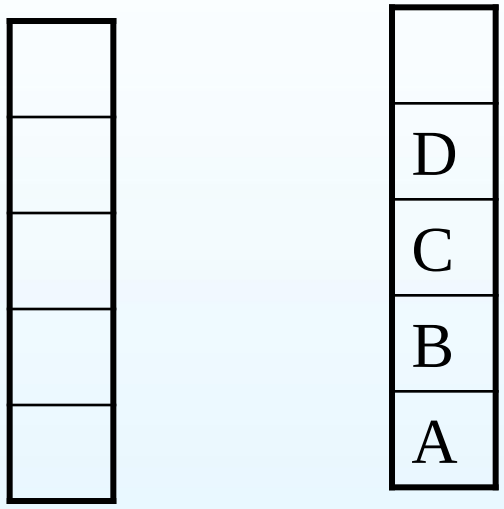
(1) 另设一个标志区别队列是空还是满；

例如：设一个变量count用来记录队列中元素个数，当count==0时队列为空，当count= MAXQSIZE时队列为满。

3.4.3 循环队列

- 循环队列空状态和满状态的讨论

空状态　　满状态



Sq.rear=0　　**Sq.rear=4**

Sq.front=0　　**Sq.front=0**

3.4.3 循环队列

□循环队列空状态和满状态的判别

(2)队满条件：以队头指针在队列尾指针的下一位置作为队列呈满状态的标志，牺牲一个存储空间；

队满条件为：

(sq.rear+1) mod maxsize==sq.front

队空条件为：sq.rear==sq.front

3.4.3 循环队列

□队列初始化算法

```
Status InitQueue (SqQueue &Q) {
                                    // 构造一个空队列Q
  Q.base = (QElemType *) malloc(MAXQSIZE *sizeof (QElemType));
  if (!Q.base) exit (OVERFLOW);
                                    // 存储分配失败
  Q.front = Q.rear = 0;
  return OK;
}
```

3.4.3 循环队列

□入队列算法

```
Status EnQueue (SqQueue &Q, QElemType e) {
                    // 插入元素e为Q的新的队尾元素
  if ((Q.rear+1) % MAXQSIZE == Q.front)
    return ERROR; //队列满
  Q.base[Q.rear] = e;
  Q.rear = (Q.rear+1) % MAXQSIZE;
  return OK;
}
```

3.4.3 循环队列

□出队列算法

```
Status DeQueue (SqQueue &Q, QElemType &e) {
  // 若队列不空，则删除Q的队头元素，
  // 用e返回其值，并返回OK; 否则返回ERROR
   if (Q.front == Q.rear)  return ERROR;
   e = Q.base[Q.front];
   Q.front = (Q.front+1) % MAXQSIZE;
   return OK;
}
```

单选题 1分

设置

若用一个大小为6的数组来实现循环队列，且当前rear和front的值分别为0和3，当从队列中删除一个元素，再加入两个元素后，rear和front的值分别为（ ）。

A 1和5

B 2和4

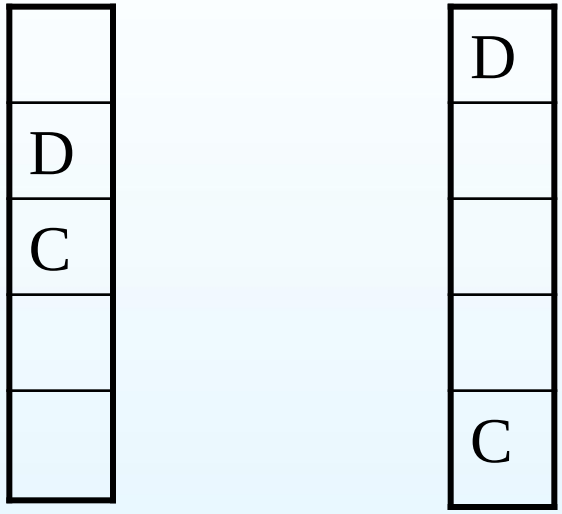
C 4和2

D 5和1

提交

3.4.3 循环队列

分析以下两种状态如何求队列长度



sq->rear=4
sq->front=2

sq->rear=1
sq->front=4

3.4.3 循环队列

□求队列长度算法

```
int QueueLength(SqQueue Q)
{
    return (Q.rear-Q.front+MaxSize)%MaxSize;
}
```

单选题 1分

设置

设循环队列的存储空间为a[0..20]，且当前队头指针（f指向队首元素的前一位置）和队尾指针（r指向队尾元素）的值分别为8和3，则该队列中元素个数为（ ）。

A 5

B 6

C 16

D 17

提交

3.4.1 双端队列

- 问题：

判断是不是回文数，例如：12345678*987654321*，

12345678987654321.

- 解决办法

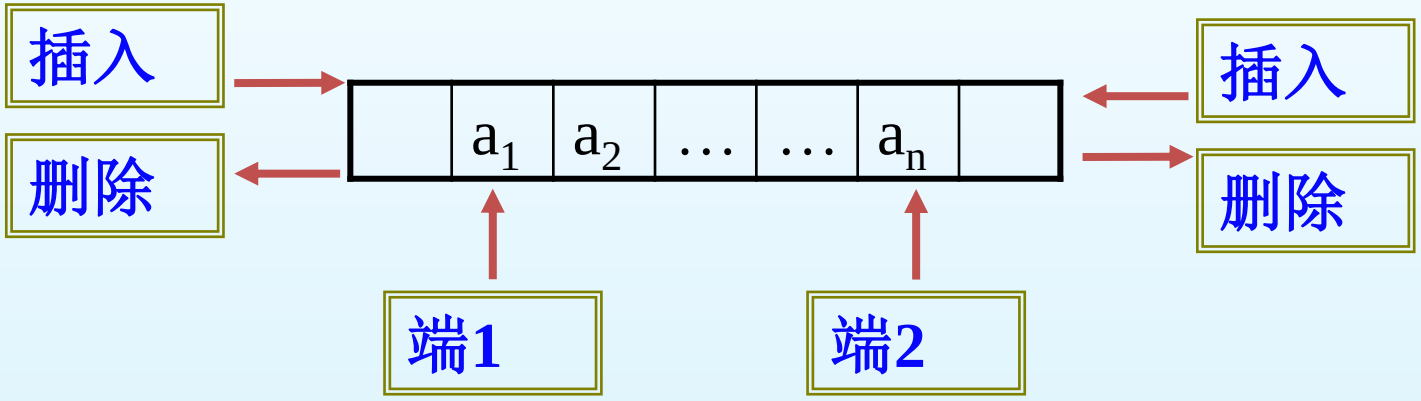
1. 设置一个栈，一个队列。
2. 如果序列长度是偶数（设为2m），则前m个依次入栈，后m个依次入队列。如果序列长度是奇数（设为2m-1），则前m-1个依次入栈，后m-1个依次入队列。
3. 当队列和栈不空时，出栈一个设为a，出队列一个设为b，如果a==b；重复3，如果a≠b；返回不是回文数。
4. 如果栈和队列全空，则是回文数，否则不是。

3.4.1 双端队列

□双端队列定义

双端队列是一种特殊的线性表，特殊在限定插入和删除操作只能在表的两端进行。

□双端队列示意图



3.4.1 双端队列

- 问题：

判断是不是回文数，例如：12345678998765432 1，
12345678987654321.

- 解决办法

1. 设置一个双端队列。
2. 把待判定序列从一端入队列。
3. 当队列不空时，分别从两端各出队元素a和b，如果a==b；重复3，如果a≠b；返回不是回文数。
4. 如果队列空或者队列中只有一个元素，则是回文数，否则不是。

◆1.线性表、栈和队列都是 _ 线性 _ _ _ _ _ 结构，可以在线性表的 _ 任何 _ _ _ 位置插入和删除元素；对于栈只能在 栈顶 _ _ _ 位置插入和删除元素；对于队列只能在队尾 _ 插入和在 队首 _ _ _ _ _ 删除元素。

单选题 1分

设置

线性表、堆栈、队列的主要区别是什么?

A 线性表用指针，堆栈和队列用数组

B 堆栈和队列都是插入、删除受到约束的线性表

C 线性表和队列都可以用循环链表实现，但堆栈不能

D 堆栈和队列都不是线性结构，而线性表是

提交

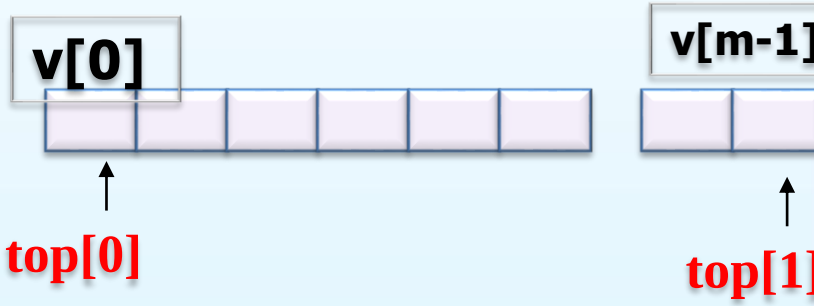
◆2.用循环链表表示的队列长度为n，若只设头指针，则出队和入队的时间复杂度分别是 O(1) _ 和 O(n)；若只设尾指针，则出队和入队的时间复杂度分别是 O(1) _ 和 O(1) 。

◆3.在具有n 个单元的循环队列中，队满时共有 n－1 个元素。

习 题

举例1、两个栈共享数组空间v[m]，它们的栈底分别设在数组的两端，每个元素占一个分量，试写出两个栈公用栈操作算法：push(i,x),pop(i)和top(i),其中i为0和1，用以指示栈号。

```
typedef struct
{ ElemType v[m];
  int top[2] ; // 栈顶指针
}STACK;
STACK S;
```



3.2 栈举例

```
void InitStack(STACK &S)
 { S.top[0]=-1;
   S.top[1]=m;
  }
```

3.2 栈举例

```
int PUSH(STACK &S,ElemType x,int i)
  { if ( S.top[1]-S.top[0])==1)
            return  error;
          else
            switch (i)
          {case 0: S.v[++S.top[0]]=x;
              break;
           case 1: S.v[--S.top[1]]=x;
              break;
           default: return error;
          }
    }
```

3.2 栈举例

```
int POP(STACK &S,ElemType &x,int i)
 { if ( i==0)
       if (S.top[0]==-1)
         return error
         else
         {x=S.v[S.top[0]]; S.top[0]--;}
    if ( i==1)
       if (S.top[1]==m)
         return error
         else
         {x=S.v[S.top[1]]; S.top[0]++;}
 }
```

单选题 1分

设置

线性表、堆栈、队列的主要区别是什么?

A 线性表用指针，堆栈和队列用数组

B 堆栈和队列都是插入、删除受到约束的线性表

C 线性表和队列都可以用循环链表实现，但堆栈不能

D 堆栈和队列都不是线性结构，而线性表是

提交

单选题 1分

设置

如果一堆栈的输入序列是aAbBc，输出为 abcBA，那么该堆栈所进行的操作序列是什么？ 设P代表入栈，O代表出栈。

A PPPOOPOPOO

B POOPPPOPOO

C POPPOPPOOO

D PPOPPOOOPO

提交

本章小结

熟练掌握：

(1)栈、队列的定义、特点和性质；

(2)ADT栈、ADT队列的设计和实现以及基本操作及相关算法。

重点学习：

ADT栈和队列在表达式求值、括号匹配、数制转换、迷宫求解中的应用，提高利用栈和队列解决实际问题的应用水平。